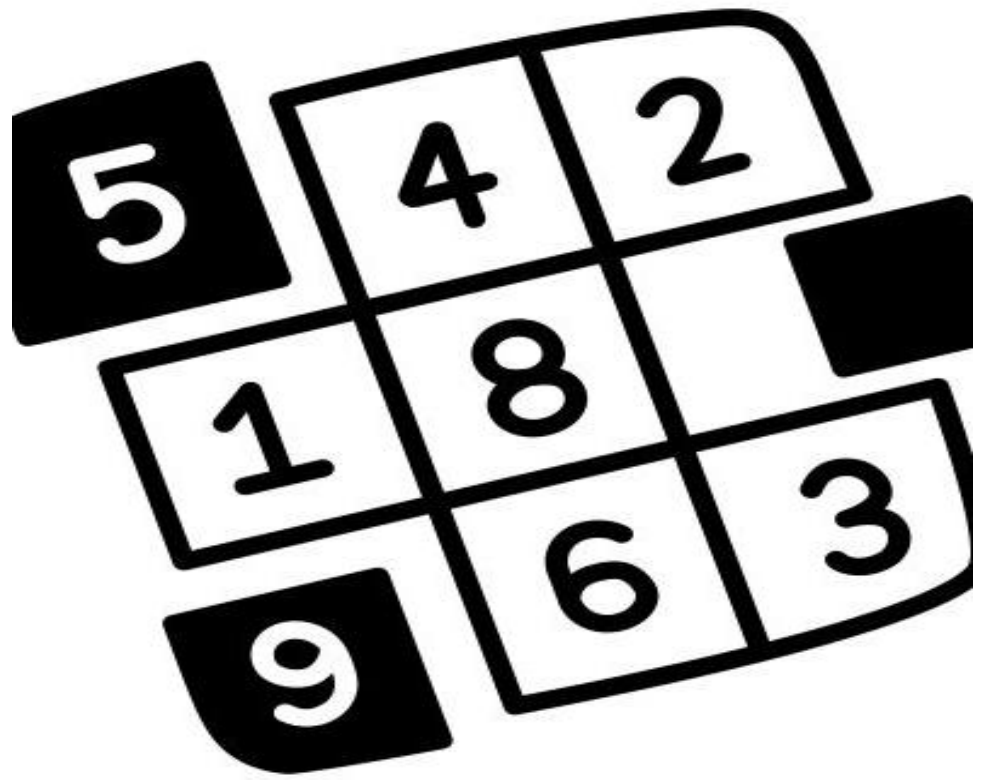




SUDOKU PROBLEM DOCUMENTATION

Made by:

Mohamed Ashraf	20230207
Mai Bakry	20230605
Mohamed Ramdan	20230315
Rana Alaa	20230207
Yusuf Hatem	20230675
Abdelrahman Alaa	20230315

1. Introduction:

The goal of this project is to implement a Sudoku solver and generator using two different programming paradigms:

- Declarative style (Functional & Recursive approach)
- Imperative style (OOP + Backtracking)

The project also includes a Graphical User Interface (GUI) built using Tkinter, allowing users to generate puzzles and display/solve Sudoku boards interactively.

2. Project Overview:

Sudoku is a logic-based puzzle that consists of a 9×9 grid divided into nine 3×3 sub-grids. The goal is to fill the grid with digits from 1 to 9 such that:

- Each row contains all digits from 1 to 9.
 - Each column contains all digits from 1 to 9.
 - Each 3×3 box contains all digits from 1 to 9.
-

3. Declarative Approach:

3.1 Overview :

The declarative version solves Sudoku using:

- Pure recursion
- Immutable grid updates
- Functional-style helpers (e.g., nested recursive functions)

3.2 Main Functions:

- **solve()** → Recursively finds all possible solutions.
- **is_valid()** → Checks Sudoku rules.
- **find_empty_cells()** → Returns empty cells recursively.
- **generate_sudoku_pair()** → Generates puzzle and solution.
- **solve_sudoku()** → Generates a valid solution using randomness.
- **remove_cells()** → Removes numbers to create a puzzle.

4. Imperative Approach:

4.1 Overview:

The imperative version is implemented using OOP and the classical backtracking algorithm.

4.2 Main Components:

Class: Board

- **Attributes:**

- board → 9×9 matrix
- code → String representation of the board

Important Methods

- **findSpaces()** → Finds first empty cell.
- **checkSpace()** → Checks row/column/box validity.
- **solve()** → Recursive backtracking solver.
- **generateRandomCompleteBoard()** → Creates a full valid Sudoku.
- **generateQuestionBoard()** → Removes numbers depending on difficulty.
- **findNumberOfSolutions()** → Ensures puzzle has *one unique* solution.
- **boardToCode()** → Converts 9×9 board into 81-char string

4.3 Difficulty Levels

- Easy → removes 36 cells
- Medium → removes 46 cells
- Hard → removes 52 cells

5. Comparison Between Declarative & imperative:

5.1 Time Complexity Analysis:

Both Sudoku solvers are based on **backtracking**, but their performance differs due to implementation style.

Declarative Solver (Functional + Recursion)

- **Time Complexity:** Worst-case $\approx O(9^n)$ where n is number of empty cells.
- Functional recursion creates **new copies of the grid** each call $\rightarrow$ slower.
- No in-place editing $\rightarrow$ **extra memory overhead**.
- recursion overhead $\rightarrow$ **deep stack + slower execution**.

Overall performance: Slowest approach.

Imperative Solver (OOP + Backtracking)

- **Time Complexity:** Worst-case $\approx O(9^n)$ (same backtracking logic).
- But uses **in-place mutation**, so each step is faster.
- No grid copying.
- Better pruning + easier optimization.

Overall performance: Faster and more efficient.

6. Conclusion:

Feature	Declarative	Imperative
Paradigm	Functional	OOP / Procedural
State	Immutable	Mutable
Performance	Slower	Faster
Code Length	Shorter	Longer
Readability	Clear logic flow	Clear structure
Ease of Debugging	Harder	Easier